

N-queens Problem by Efficient Non-backtracking Algorithm Using Local Search, Heuristics and Tabu Search Elements

Marian Kovac

Faculty of Management science and Informatics,

University of Zilina

Zilina, Slovakia

marian.kovac@fri.uniza.sk

Abstract—This paper presents an efficient algorithm using Local search, various heuristics and Tabu search elements, which is capable to find solution for huge instances of the *n*-queens (hundreds of millions *n*). The algorithm returns random solutions in short time even on an ordinary personal computer. There is no other faster algorithm in *n*-Queens bibliography as our presented algorithm so far.

I. INTRODUCTION

The *n*-queens problem is standard and well-known combinatorial problem that consists of placing *n* queens on a $n \times n$ chessboard, such that they do not attack each other within standard chess queens' moves. We can formulate the *n*-queens problem as follows: There is $n \times n$ board on which *n* queens must be placed such that on every row and every column there is exactly one queen and on any diagonal is at most one queen. The *n*-queen problem is constraint satisfaction problem (CSP), what means it is defined by a set of variables (positions of all *n* queens), values (all possible emplacements of queens) and a set of constraints (exactly one queen in every row and column, at most one queen at arbitrary diagonal). Each constraint excludes some possible combinations of assignment of values to variables [4]. The goal is to find complete assignment (every variable has assigned a value) that satisfies all the constraints (in other words, minimize number of conflicts among the queens).

There are many practical applications of the *n*-queen problem to the real world problems, such as real-time systems, computer resource management, VLSI circuit design, VLSI testing, task scheduling, traffic control, deadlock prevention, data/message routing, task assignment, load balancing in multiprocessor computers, data compression, image processing, low-density parity-check codes, communication system design, optical parallel processing, parallel memory storage schemes and so on [1], [2], [3]. Therefore the effort spent on analysis and solving of the *n*-queen problem is valuable indeed.

Many approaches to solving the *n*-queens problem are based on backtracking search where queens are added to some positions of the board gradually in regard to various rules [9]. When there is no place for the next queen, one or more queens are removed from the board and again added gradually on different positions than before. This process is repeated till all queens are placed on the board. More sophisticated backtracking searches use heuristics to decide where to place the next queen (e.g. place the next queen on such position, that the greatest amount of other non-attacked positions on the board is guaranteed). Another group of approaches is based on various meta-heuristics, such as Genetic algorithms [10], Ant Colony Optimization [5], Particle Swarm Optimization [3], Simulated Annealing [8]... Despite of many ingenious methods for solving the *n*-queen problem [7] they are able to find the solution only for small *n*. This paper presents the algorithm focused on finding the solution of the greatest possible instances of the problem, with the lowest possible time and memory consumption. Presented algorithm works in linear time and it returns one solution (emplacement of queens) after its run.

The rest of the paper is organized as follows: in section II. the proposed algorithm for solving the *n*-queens problem is presented and closely described. Computational results are summarized in section III. Section IV. concludes the paper.

II. THE ALGORITHM

The emplacement of queens on the board can be described by a *n*-dimensional array, elements of which are elements of the set $\{1, 2, \dots, n\}$ (this array will be denoted as *Q*). That means, elements of *Q* are always a permutation of the set $\{1, 2, \dots, n\}$. Subsequently, value on *k*-th position of *Q* represents column number in which the queen is placed in *k*-th row (e.g. the emplacement of queens in figure 1 can be represented by $Q = (2, 4, 1, 3)$). Hence it follows that the complexity of the *n*-queens problem is $O(n!)$. Thus, a heuristic approach must be used for obtaining results in reasonable time.

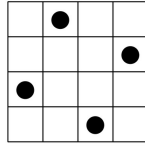


Fig. 1. The emplacement of queens on the board

Such representation of the emplacement of queens (permutation of the set $\{1, 2, \dots, n\}$) guarantees that no more than one queen will be in every row and every column. It means there will be always only one type of conflicts among the queens - conflicts on the diagonals (two or more queens on the same diagonal). We count as conflict every pair of queens on the same diagonal (i.e. four queens on the same diagonal means six conflicts). Then total number of conflicts (C_d) on diagonal d can be computed as follows:

$$C_d = \frac{q \cdot (q - 1)}{2},$$

where q is amount of queens on diagonal d .

Our early experiments were based on backtracking search with various modifications and improvements. But this direction wasn't very successful at reaching our goal at all (the algorithm was able to find solution only for very small n in acceptable time). Thus, there was a need to change the approach and we decided to start from randomly generated emplacement of the queens, whereby we focused on conflicts elimination (figure 2).

The algorithm consists from two successive parts, which

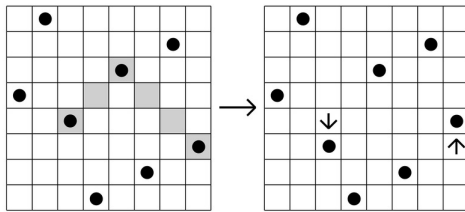


Fig. 2. Conflicts elimination among queens

will be denoted as 1. *Pre-shaking* and 2. *Shaking* (all functions and variables used in pseudo-codes will be explained after the description of the algorithm).

1. Pre-shaking

In the first part, the algorithm gradually walks through rows on the board and exchanges the rows so that overall amount of conflicts is decreased (Local search).

- 1) Place all queens on the board randomly (uniform distribution) such that there is exactly one queen in every row and every column (regardless of whether there are any conflicts among the queens or not). Like it was said above, the emplacement of queens on the board is represented by a n -dimensional array Q .

```
for (i = 1 to n) do:
    Q[i] = i
shuffle(Q)
```

During 18 000 runs of the algorithm it was observed that average number of conflicts among the queens after this step of the algorithm is $\frac{2}{3}n$ approximately (i.e. if $n = 1000$, there is approximately 667 conflicts among queens in average after this step).

- 2) Gradually walk through rows of the board and check, whether the queen in the actual row (i -th row) is causing any conflict with queens in rows $1, 2, \dots, (i-1)$. If queen in the actual row is not causing conflicts, go to the next row ($i+1$). If queen in the actual row is causing one or more conflicts, scan next rows ($i+1, i+2, \dots, n$) one by one and search for such row, which will not cause any conflict with queens in rows $1, 2, \dots, (i-1)$ after its exchange with actual (i -th) row. If such row is found, perform the exchange. If such row isn't found, leave actual row without change and advance to the next row ($i+1$).

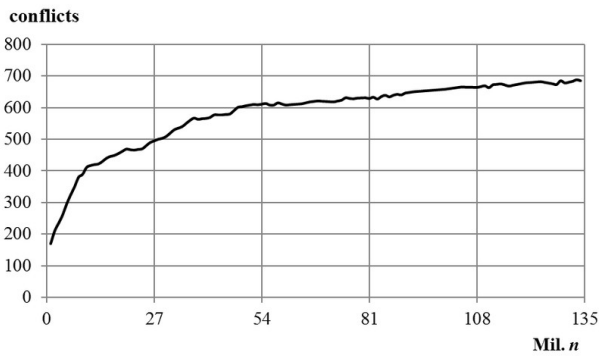
```
i = 1
while (i < n) do:
    if (num_conflicts(1..i) = 0) do:
        inc(i)
    else do:
        j = i + 1
        exchange_rows(i, j)
        while (num_conflicts(1..i) > 0
            and j <= n) do:
            exchange_rows(i, j)
            inc(j)
            exchange_rows(i, j)
        inc(i)
```

The second step of the first part of the algorithm is able to reduce overall amount of conflicts significantly. Initial amount of conflicts ($\approx \frac{2}{3}n$) obviously associated with n is reduced to a constant just slightly affected by n . Average amount of conflicts after *Pre-shaking* part is shown in figure 3.

In order to decrease overall amount of conflicts even more, we experimented with multiple application of the second step of the first part of the algorithm. During experiments, we rotated/flipped the board after every procedure (application of the second step) and subsequently applied the procedure again. We experimented with rotation by 90, 180 and 270 degrees of the board and with vertical flip of the board. We tried various combinations of rotations and flips, but it decreased overall amount of conflicts just in very few cases. In most of cases overall amount of conflicts was gently increased. So we use only one application of the procedure in the final algorithm.

2. Shaking

The aim of the second part of the algorithm is to eliminate all remaining conflicts among the queens completely,

Fig. 3. Average amount of conflicts among queens after *Pre-shaking* part

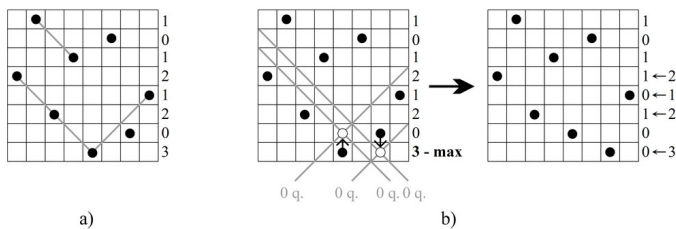
using probabilistic Local search, heuristics and Tabu search elements.

- 1) To every row assign the value equal to number of conflicts which causes the queen in the given row (values at right side of the boards in figure 4).
- 2) Heuristics: mark rows with maximal number of conflicts (assigned in the first step) as $m_1, m_2, \dots, m_k$ (e.g. there will be only one row m_i in figure 4 a, namely $m_1 = 8$). Gradually walk through rows m_i and search for such row for which the exchange with row m_i ensures migration of queens to the diagonals with no other queens (let's denote such diagonals as "*empty diagonals*"). If two such rows are found (figure 4 b - rows 7 and 8), perform the exchange (what guarantees that total amount of conflicts on the board is decreased) and continue with step 1. The exchange is performed only if it is not prohibited by *Tabu list*. Using *Tabu list* protects the algorithm from looping (as it will be shown below the description of the algorithm).

```

index_array[] = 1..n
found_empty_diags = FALSE
for each m_i do:
  j = 1
  shuffle(index_array)
  while (
    queens_on_diags(m_i, index_array[j]) > 0 and
    j <= n or
    is_tabu(Q, m_i, index_array[j])
  ) do:
    inc(j)
  if (j <= n) do:
    exchange_rows(m_i, index_array[j])
    found_empty_diags = TRUE
    exit

```

Fig. 4. Solving the n -queens problem

- 3) If diagonals with no other queens (*empty diagonals*) were not found (the variable *found_empty_diags* is set

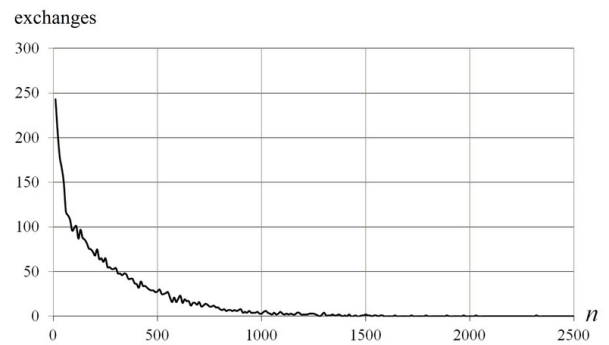
to *FALSE*), use following heuristics: Gradually walk through rows m_i and try to exchange the actual row (m_i) with several other rows. The number of the exchanges is set by variable *tries*. If overall number of conflicts after the exchange decreases, perform the exchange, otherwise continue without changes. The exchange is performed only if it is not prohibited by *Tabu list*.

```

index_array[] = 1..n
local_extrema = TRUE
exchanges = 0
for each m_i do:
  j = 1
  shuffle(index_array)
  while (j <= tries) do:
    A = total_conflicts(Q)
    while (
      is_tabu(Q, m_i, index_array[j])
    ) do:
      inc(j)
    exchange_rows(m_i, index_array[j])
    B = total_conflicts(Q)
    if (B >= A) do:
      exchange_rows(m_i, index_array[j])
    else do:
      local_extrema = FALSE
      exit
    inc(j)
  exchanges = exchanges + j - 1

```

This step is like a "preventer" for cases, which are not covered by heuristics in previous step of the algorithm, since it assumes sufficiently large dimension of the board for occurrence of *empty diagonals*. But for $n < 1500$ approximately the occurrence of *empty diagonals* is not sufficient and just in these cases acts this step of the algorithm. Figure 5 shows average number of tried

Fig. 5. Average number of tried exchanges performed by third step of part *Shaking* (value of variable *exchanges* after the solution is found)

exchanges performed by this step of the algorithm (value of variable *exchanges*) to find the solution for various n . This step of the algorithm isn't performed at all for $n > 2000$ as it is obvious from the figure.

- 4) If the algorithm was unable to perform the exchange, which would decrease the number of total conflicts among the queens (in previous two steps), we suppose we got stuck in local extrema. Subsequently we accept such exchange of already performed exchanges (also in previous two steps), which ensures the lowest increase of total conflicts. That is the way the algorithm uses to escape from the extrema.
- 5) Continue with step 1.

Notice, there are mentioned only essential steps in the description of the algorithm and in pseudo-codes, substantial for the understanding principle of the algorithm. The algorithm itself naturally contains many partial steps and optimizations, which inclusion into the description isn't necessary in order to understand principle of the algorithm. Some steps included in listed pseudo-codes are only illustrative and they are not in the algorithm actually or they are in the algorithm in other more complicated form.

Functions and variables used by the algorithm:

- function **shuffle(array)** shuffles elements of the *array* randomly,
- function **num_conflicts($i..j$)** returns number of conflicts, caused by queen in row j with queens in rows $i, i+1, \dots, j-1$,
- function **exchange_rows(i, j)** exchanges queens in rows i and j ,
- function **inc(i)** increments variable i by one (equal to command $i = i + 1$),
- command **index_array[] = 1..n** fills array *index_array* with values $1, 2, \dots, n$,
- function **queens_on_diags(i, j)** returns amount of queens occurring on the diagonals, which will be occupied by queens in the rows i and j after their exchange (figure 4 b),
- function **total_conflicts(Q)** returns total number of conflicts among the queens on the board, represented by Q . Number of conflicts is equal to number of different pairs of the queens situated on the same diagonal (total amount of conflicts on the board in figure 4 a is 5 and total amount of conflicts on the board in figure 4 b is 2),
- function **is_tabu(Q, i, j)** returns Boolean value *TRUE*, if given exchange of rows i and j is evaluated as unacceptable. The exchange of rows is evaluated as unacceptable in three cases:
 - if given row should be exchanged with itself;
 - if there should be exchanged the same rows, of which directly former exchange rose actual configuration of queens;
 - if examined exchange of rows should cause looping (table I - the exchange of queens on relevant positions in row marked with "X" is rejected). The algorithm utilizes *Tabu list* with fixed length, set to four last exchanges of queens for purpose of loop prevention.

turn	position of queens (Q)					exchange of positions	
1.	1	<u>2</u>	3	<u>4</u>	5 6	2 - 4	
2.	1	4	3	<u>2</u>	<u>5</u> 6	4 - 5	
3.	1	<u>4</u>	3	<u>5</u>	2 6	2 - 4	
4.	1	<u>5</u>	3	4	2 6	2 - 5	X
5.	1	<u>2</u>	3	<u>4</u>	<u>5</u> 6	2 - 4	
6.	...					...	

TABLE I
THE LOOPING AT THE EXCHANGE OF QUEENS

No other circumstances causing looping were observed during 18 000 runs of the algorithm.

- variable **tries** limits total number of efforts of exchange one of the rows m_i causing maximal amount of conflicts (e.g. last row of the board in figure 4 a) with any other row (which is not excluded by *Tabu list*). It was observed that the algorithm gains the best results with value of variable **tries** from $\{10, 18\}$.

III. IMPLEMENTATION OF THE ALGORITHM AND COMPUTATIONAL RESULTS

The algorithm was implemented in C++ (standard C++11/C++0x) and ran on computer with Mobile Intel i7-2630QM (2 GHz) CPU, with 3.7 GB available memory approximately (it is obvious that by using more powerful computer it is possible accelerate the search significantly and we can find the solution for greater n also).

Average execution time of the algorithm for various n is listed in table II. The graph of execution times is shown in figure 6. Listed values were gathered from 18 000 runs of the algorithm and with variable *tries* set to value 10 (as well as at other figures and tables). Memory consumption of the algorithm for finding the solution for 134 000 000 queens is 3.72 GB.

Queens (n)	Execution time	Queens (n)	Execution time
< 1 000 000	< 1 s.	70 000 000	$\approx$ 26 s.
5 000 000	$\approx$ 2 s.	80 000 000	$\approx$ 31 s.
10 000 000	$\approx$ 3 s.	90 000 000	$\approx$ 35 s.
20 000 000	$\approx$ 7 s.	100 000 000	$\approx$ 38 s.
30 000 000	$\approx$ 10 s.	110 000 000	$\approx$ 42 s.
40 000 000	$\approx$ 14 s.	120 000 000	$\approx$ 45 s.
50 000 000	$\approx$ 18 s.	132 000 000	$\approx$ 50 s.
60 000 000	$\approx$ 23 s.	134 000 000	$\approx$ 51 s.

TABLE II
EXECUTION TIME OF THE ALGORITHM - AVERAGE OF 18 000 RUNS

IV. DISCUSSION AND CONCLUSION

Presented algorithm works in linear time (i.e. linear increase of execution time along with linear increase of number of queens). It uses Local search, various heuristics and Tabu search elements, which all ensures that the algorithm works efficiently, reliably (18 000 runs of the algorithm without any significant deviation of execution time or failed search - the

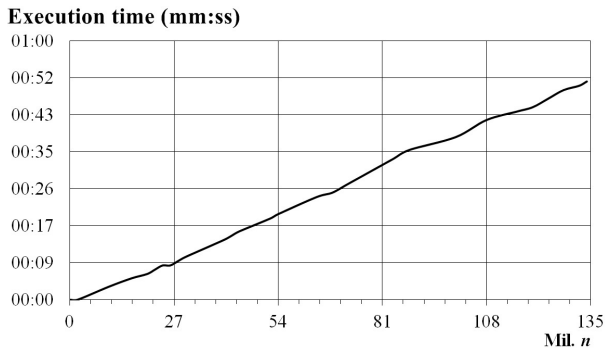


Fig. 6. Graph of execution time of the algorithm for the n -queen problem

algorithm is stable) and it doesn't use any form of backtracking, not even repeated search. It means that every time that the algorithm is launched, it returns the solution without an interruption of search and starting from the beginning and also without backtracking to previous states. Due to probabilistic Local search used in 2. *Shaking*, the algorithm returns different solutions.

In addition, there is no other algorithm listed in n -Queens bibliography [6] (currently with 332 references) able to find solution for the n -queens problem for such huge scale as our algorithm and simultaneously there is no other algorithm as fast as ours along with small memory consumption. That all opens new possibilities in two directions: in solving of real world problems, which is the n -queens problem applied to; and in proposing algorithms for solving other constraint satisfaction problems, where our presented ideas can be utilized.

REFERENCES

- [1] J. Bell, B. Stevens, A survey of known results and research areas for n -queens, *Discrete Mathematics* 309 (2009), pp. 131
- [2] D. Du, J. Gu, P.M. Pardalos, Satisfiability Problem: Theory and Applications, in *Proceedings of a DIMACS Workshop*, vol. 35, March 11-13, 1996
- [3] X. Hu, R.C. Eberhart, Y. Shi, Swarm Intelligence for Permutation Optimization: A Case Study of n -Queens Problem, in *Swarm Intelligence Symposium, 2003. SIS '03. Proceedings of the 2003 IEEE*, pp. 243-246, 24-26 April 2003, ISBN: 0-7803-7914-4.
- [4] T. Hulubei, B. O'Sullivan, Optimal Refutations for Constraint Satisfaction Problems, in *IJCAI-05, Proceedings of the Nineteenth International Joint Conference on Artificial Intelligence*, Edinburgh, Scotland, UK: Professional Book Center, 2005, pp. 163-168, ISBN: 0938075934.
- [5] S. Khan, Solution of n -Queen Problem Using ACO, in *Multitopic Conference, 2009. INMIC 2009. IEEE 13th International*, pp. 1-5, Islamabad, 14-15 Dec. 2009, ISBN: 978-1-4244-4872-2.
- [6] W. Kusters, n -Queens bibliography, in <http://www.liacs.nl/~kusters/n-queens>, 23.5.2013.
- [7] I. Martinjak, M. Golub, Comparison of Heuristic Algorithms for the N -Queen Problem, in *Proceedings of the ITI 2007 29th Int. Conf. on Information Technology Interfaces*, June 25-28, 2007, Cavtat, Croatia, pp. 759-764.
- [8] V.M. Saffarzadeh, P. Jafarzadeh, M. Mazloom, A Hybrid Approach Using Particle Swarm Optimization and Simulated Annealing for N -queen Problem, in *World Academy of Science, Engineering & Technology*; July 2010, Vol. 67, pp. 974-978.
- [9] P. San Segundo, New decision rules for exact search in N -Queens, in *Journal of Global Optimization*, Volume 51, Issue 3, pp. 497-514, Springer US, 2011, Print ISSN: 0925-5001, Online ISSN: 1573-2916.

- [10] A.M. Turkey, M.S. Ahmad, Using Genetic Algorithm for Solving N -Queens Problem, in *Information Technology (ITSim), 2010 International Symposium*, vol. 2, pp. 745-747, Kuala Lumpur, 15-17 June 2010, ISSN: 2155-897.